

DOCUMENTED OUTCOMES. MEASURABLE RESULTS. REAL SYSTEMS.

Engineering Case Studies

— Proof over promises. —

Full-Stack Engineering

Next.js · React · TypeScript · FastAPI
· PostgreSQL

Autonomous AI Workflows

LLM orchestration · RAG pipelines ·
Playwright RPA

Robotic Process Automation

Headless bots · 24/7 execution ·
legacy systems

Transparent Pricing

Published rates. Free prototype
before any payment.

Engineering-First

The founder writes production
code on every project.

Deterministic AI

Strict JSON scaffolding on every
LLM integration.

8+

Platforms

Omni-channel publishing

100%

Custom-Built

No templates. Ever.

₹8,999

Starting From

Digital Launch — 4 weeks

COMPANY

Logic Intelligence Technologies

FOUNDER

Vikash Saravanan

LOCATION

Coimbatore, Tamil Nadu, India

WEBSITE

logicintelligencetechnologies.in

“Where Logic Meets Innovation” — every system we architect, every line we deploy.

Visit logicintelligencetechnologies.in to explore services, case studies, and pricing packages.

CORE SERVICES

Full-Stack Web Apps

AI & LLM Integration

Robotic Process Automation

PostgreSQL & Supabase

CI/CD & DevOps

E-Commerce Platforms

RAG Pipelines

API Engineering

SECTION CS-01

OpenEnv-Debugger

Meta PyTorch Global Hackathon · AI-Powered CLI Debugging Tool

Category: AI Engineering · Stack: Python · Claude API · PyTorch · CLI

THE CHALLENGE

ML engineers lose hours to cryptic PyTorch installation errors — CUDA version mismatches, wrong Python environment, missing packages, conflicting dependencies. Stack Overflow gives contradictory advice for different OS versions. No tool existed that could read the error, understand the environment, and prescribe the exact fix.

THE SOLUTION

OpenEnv-Debugger is a Python CLI tool that reads PyTorch error logs and uses Claude AI with strict JSON schema output to diagnose the root cause and generate step-by-step resolution commands. Deterministic AI engineering: structured prompt validated JSON response typed resolution object CLI output. No hallucination risk. Every response validated before display.

KEY TECHNICAL DECISIONS

- ◆ JSON mode enforced on every Claude API call — malformed responses trigger auto-retry
- ◆ Pydantic v2 schema validates diagnosis object structure before CLI rendering
- ◆ argparse CLI with --verbose flag for raw API response inspection
- ◆ Environment introspection via subprocess: reads pip freeze, python --version, nvidia-smi automatically
- ◆ GitHub Actions CI pipeline with automated test suite on every push

MEASURABLE OUTCOMES

4 min avg

Time to resolution vs 2-4 hrs manual

100%

Structured JSON output on all valid errors

Hackathon

Meta PyTorch Global submission

SECTION CS-02

HearWise Child Health

Production Healthcare SaaS · Supabase RLS · Next.js · TypeScript

Category: Full-Stack Healthcare Platform · Status: Live in Production

THE CHALLENGE

Paediatric health platforms must handle sensitive PII under strict access controls. Off-the-shelf platforms cannot enforce per-parent data isolation at the database level. Every row in every table must be visible only to the user who owns it — even if the API is compromised, the data must remain inaccessible.

THE SOLUTION

HearWise implements Row Level Security at the PostgreSQL layer in Supabase. Every table has an RLS policy written in SQL that checks the JWT token's user ID against the row's owner_id column before any query is allowed to return data. No application-layer check can override this — it is enforced by the database engine itself. Combined with Next.js SSR, Supabase Auth, and a Lighthouse 96 accessibility score, HearWise sets the standard for secure paediatric digital health.

TECHNICAL ARCHITECTURE

Auth Layer	Supabase Auth with Email/Password and Magic Link. JWT tokens include user ID claim validated on every RLS policy.
RLS Policies	CREATE POLICY per table. auth.uid() = owner_id condition. Applied to SELECT, INSERT, UPDATE, DELETE.
Frontend	Next.js 15 with SSR. Supabase client hydrated server-side. TypeScript strict mode. Tailwind dark theme.
Performance	Lighthouse 96 accessibility. Core Web Vitals green. Vercel Edge Network for global sub-100ms response.

SECTION CS-03

Omni-Apply RPA Bot

Autonomous Job Application Bot · Playwright · Python asyncio · LLM

THE CHALLENGE

Job seekers spend 30–40 hours per week filling identical application forms across dozens of portals. Each portal has a unique interface, unique field names, unique authentication flow. No single automation tool handled all major Indian job platforms simultaneously.

THE SOLUTION

Playwright headless browser bots run in parallel via Python asyncio coroutines. Each bot handles one portal: login, session persistence, DOM parsing, AI-assisted form field identification, and form submission. An LLM reads the portal's HTML and maps fields to the user's profile JSON. The orchestrator dispatches bots to 100 portals simultaneously. 40 hours of manual application work completes in under 4 minutes.

RESULTS

40 hrs 4 min

Application time reduced

100+ Portals

Simultaneously active bots

0 Manual Steps

After initial profile setup

24/7 Operation

Scheduled, cloud-deployed

CONTACT & ENGAGEMENT

Website logicintelligencetechnologies.inEmail support@logicintelligencetechnologies.in

Phone +91 93428 77474

Location Coimbatore, Tamil Nadu, India